WARN:
This document is still under development and will be completed around 1.2.2027

PROZESSOR

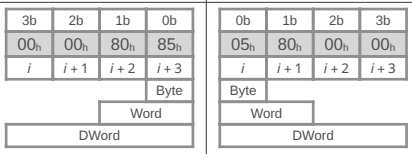
Adressbus: Adresse der zuzugreifenden Speicherzelle
Datenbus: Daten, die aus Speicherzelle gelesen wurde/geschrieben werden soll
Steuersignale: Geben an, was getan werden soll
Register: Speicher im Prozessor
Operation: Prozessorspezifisch, haben 0..*n* Operanden
Instruktion: Kombination von Operation + Operanden
Sequenzen: Hart-codiert im Prozessor oder Hauptspeicher
Zyklus: Prozessor, über mehrere Takte hinweg:
1) Fordert Instruktion ab Adresse vom IP an
2) Decodiert Operation und Operanden aus Instruktion
3) Wählt mit Operation korrespondierenden Baustein aus
4) Aktiver Baustein liest ggf. aus den Registern
5) Aktiver Baustein führt ggf. Berechnung aus
6) Aktiver Baustein schreibt ggf. in die Register
7) Erhöht Befehlszeiger entspr. der Länge der Instruktion

MASCHINENCODE

Begriff	*Definition*
Byte	8 Bit
Word	2 Byte / 16 Bit
Doubleword	4 Byte / 32 Bit, auch DWord
Quadword	8 Byte / 64 Bit, auch QWord
Double Quadword	16 Byte / 128 Bit, auch DQWord

Kleinste Einheite ist ein Byte (8 Bit). Ein einzelnes Bit muss immer mit dem gesamten Byte verarbeitet werden.

Big-Endian	*Little-Endian*
Von rechts nach links ← Zahlen beginnen mit dem «grossen Ende», dem ***MSByte***	Von links nach rechts → Zahlen beginnen mit dem «kleinen Ende», dem ***LSByte***



ASSEMBLY

Traditionell arbeitet der NASM im 16-Bit-Modus, Operationen haben hier andere Opcodes als im 64-Bit-Modus. `BITS 64`

Aktueller Offset: (Adresse) $
Label: Werden nicht in Maschinencode übersetzt
Flat-Form binaries: Bytesequenz ohne Zusatzinformationen
Objekt-Dateien: Beinhalten Metadaten, u.a. Symboltabelle
Exportierte Symbole: mit `global` deklarierte Label
Importierte Symbole: mit `extern` deklarierte Label, haben keinen definierten Wert innerhalb der Objekt-Datei

SYMBOLTABELLE

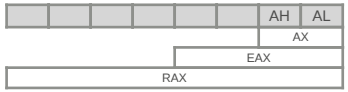
```
global x, y
extern z
dq 0, 0
w: dq 0x12345678
x: dq 0xCAFEFACE
y: dq z
```

Offset	Attribute (l)ocal (g)lobal,	Sektion	Zusatzinformationen	Name
00000010	l	.text	00000000	w
00000000		*UND*	00000000	z
00000018	g	.text	00000000	x
00000020	g	.text	00000000	y

Als extern deklariertes Label `z` erscheint als `*UND*` (undefined)

REGISTER

General Purpose: Kann nach belieben verwendet werden
Special Purpose: Wird für einen spezifischen Zweck verwendet, nur gewisse Operationen können darauf zugreifen



ALLZWECKREGISTER (GPR)

RAX: Accumulator, für einige Rechenop. das einzig mögliche
RCX: Counter für Schleifen und Stringoperationen
RDX: Pointer für I/O-Operationen
RBX: Datenpointer
RSI, RDI: Quell- und Zielindizes für Stringoperationen
RSP: Stackpointer, Adresse des allozierten Stacks
RBP: Basepointer, Basis des Stackframe der Funktion
R8 - R15: Zusätzliche Register

SPEZIALREGISTER (SPR)

RIP: Instruction Pointer, zeigt auf die nächste Instruktion
SS, CS, DS, ES, FS, GS: Segment Register
RFLAGS: Flags die spezifische Situationen markieren
CR0 - CR15: Control-Registers um die CPU zu konfigurieren
GDTR: Enthält die Adresse der GDT - verwendent für VM
LDTR: Enthält die Adresse der LDT - verwendent für VM
IDTR: Enthält die Adresse der IDT - verwendent für VM
TR: Enthält die Adresse des TSS - für Task Switching

64-Bit ISA (SPL, BPL, SIL und DIL) Instruktionen haben REX Präfix. Bsp: `mov ah, sil` entspricht `REX mov ah, sil`

DATENTRANSFER-OPERATIONEN

`mov ziel, quelle`
Nach Ausführung der Operation enthalten Quelle und Ziel den gleichen Wert.

In ein ***Register*** kann man kopieren:

Woher	*Befehl*	*Auswirkung*
Andere Reg.	`mov rax, rbx`	Inhalt `rax`=Inhalt `rbx`
Eine Konstante	`mov rax, 0x8000`	Inhalt `rax`=8000h
Vom Speicher	`mov rax, [0x8000]`	Inhalt `rax` =Inhalt `800[0..7]h`

In den ***Speicher*** kann man kopieren:

Woher	*Befehl*	*Auswirkung*
Andere Reg.	`mov [0x800], rbx`	Inhalt `800[0..7]h` =Inhalt `rax`
Konstante	`mov qword [0x8000], 5`	Inhalt `800[0..7]h`=5

aber **nicht** direkt vom Speicher in den Speicher.
Operandengrössen werden mit `byte,word,dword,qword` angegeben. Operanden müssen gleich gross sein.

ADRESSIERUNG

Displacement: `[a]` Adresse `a` der Stelle folgt unmittelbar
```
mov rax, [0x8000]
```
Base (Register): `[r]` Adresse der Stelle steht im Register `r`
```
mov rbx, 0x8000
mov rax, [rbx]
```
Scaled Index: `[i*s]` Adresse `i*s` besteht aus Index `i` (Register) skaliert Konstante `s` (1,2,4 oder 8)
```
mov rcx, 0x1000
mov rax, [rcx * 8]
```
Jede Summe der drei vorherigen: Alle drei Adressierungsmodi können beliebig addiert werden
```
mov rbx, 0x4000
mov rcx, 0x1000
mov rax, [0x2000 + rbx + rcx * 2]
```

LINKER

Assembler erzeugt aus jeder `asm`-Datei eine Objekt-Datei. Der Linker `ld` erstellt daraus ein Executable. Standard-Einsprungspunkt: `_start`:

SYSCALLS

`syscall` übergibt die Ausführung an das OS. In `rax` übergibt man den Code, in `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9` die Parameter.

```
mov rax, 60     ; syscall exit
mov rdi, 0      ; exit code
syscall
```

C

`.c` Präprozessor `.c` Compiler `.asm` Assembler `.o` Linker `.exe`

Präprozessor: Kommentare, macros, directives, tokenization. Mehrere Durchläufe, jedes mal gesamte Datei einmal.

TODO:
tokens?

`#include <file.h>` sucht nur in den Systemverzeichnissen
`#include "file.h"` erst aktuelles Verzeichnis, dann System
Wird alles zu einer ***Translation Unit*** zusammengeführt.
Taucht eigener Makroname in der Ersetzung auf, wird er nicht ersetzt, um infinite Rekursion zu verhindern.

Compiler: übersetzt eine C-Datei in eine Assembler-Datei.

Deklarationen haben keinen direkten Einfluss auf den erzeugten Byte-Stream (ähnlich wie Label in Assembler).
✓ Mehrfache deklaration (solange gleiche Typsignatur)
× Mehrfache definition

VARIABLEN

global: Speicher wird fix reserviert, die ohne Initialwert werden mit 0 initialisiert, standardmässig exportiert: `int c = 5;` entspricht in Assembler
```
global c
c: dd 5
```
extern: Globale Variablen, die aus anderen Objekt-Dateien verwendet werden sollen: `extern int c;` entspricht in Assembler `extern c`
static: Variablen, die nicht exportiert werden sollen: `static int c = 5;` entspricht in Assembler: `c: dd 5`

Bez. .o	*Dekl. .asm*	*Bez. .c*	*Dekl. .c*
lokal		global internal Linkage	static
global	global	global external Linkage	
	extern	extern	extern

***Beispiel** 1:*
```
// a.c              // Kompiliert und Linkt, kann aber
extern int x;       // crashen. Bestenfalls passiert
// b.c              // ein segfault. → in .h Files
short x;            // definieren, überall inkludieren
// c.c
static int x = 4;   // Compiler legt Speicher an
extern int x;       // extern int x macht nichts
```

Objekt: Ein zusammenhängender Speicherbereich. Inhalt kann verschieden interpretiert werden.
Wert: Genaue Bedeutung des Inhalts, wenn Typisiert

TYPEN

C kennt verschiedene Arten von Typen:
– Basistypen
– Abgeleitete Typen
– Enumerationen

Jeder Typ `T` hat feste Objektgrösse `sizeof(T)`, wird in Vielfachen von `char` angegeben, genaue Grösse in `CHAR_BIT` in `<limits.h>`. Bsp: `sizeof(T) * CHAR_BIT`
Bei Operationen erzwingt C Typgleichheit der Operanden. Nach Übersetzung in Assembler gibt es keine Typen mehr.

Compiler kann statische checks durchführen, richtige Grösse für Objekte allozieren, richtige Operationen auswählen, z.B. unsigned/signed.

Typen hängen in C von der verwendeten Maschine (und dem Compiler) ab, insbesondere ihre Grösse.
Vorteil: C-Programme laufen überall möglichst effizient
Nachteil: Plattform-Unabhängigkeit schwierig Knowhow

Typ	*Grösse*
`char`	≥ 8 Bit
`short int`	≥ 16 Bit
`int`	≥ 16 Bit ∧ ≥ `short`, «natürliche» Grösse der Arch.
`long int`	≥ 32 Bit ∧ ≥ `int`
`long long int`	≥ 64 Bit ∧ ≥ `long`

TODO:
char/signed/unsigned?

Dezimalzahl: Folge von Ziffern (`0..9`), die **nicht** mit `0` beginnt
Oktalzahl: Folge von Ziffern (`0..7`), die mit `0` beginnt
Hexadezimalzahl: Folge von Ziffern (`0..F`), die mit `0x` beginnt

Typen-Suffixe können angehängt werden: `l,ll,u,ul,ull`. Inferrierte Typ ist der kleinste, in den Konstante passt, mind. `int`

Expliziter cast: `int x = 5; long y = (long) x;`

ARITHMETISCHE UND LOGISCHE OPERATIONEN

Alle Operationen können ausschliesslich aus NAND dargestellt werden. Sie sind leicht als Transistoren zu realisieren.

LOGISCHE OPERATIONEN

ASSEMBLER

Auf Intel 64 ist das Ziel gleichzeitig ein Operand:

```
not z     =  z ← ¬z
and z, q  =  z ← z ∧ q
or  z, q  =  z ← z ∨ q
xor z, q  =  z ← z ⊕ q
```

Bitweise Operatoren agieren ohne Interpretation der Operanden als Gesamtzahl (signed und unsigned).

Bei manchen Instruktionen kann die Grösse nicht abgeleitet werden, z.B. `not [myvar]`. In diesem Fall muss man einen size specifier hinzufügen, z.B. `not word [myvar]`

C

Bitweise Operatoren entsprechen direkt den Assembler-Ops:

```
x =  ~ z  =  x ← ¬z
x = z & q  =  x ← z ∧ q
x = z | q  =  x ← z ∨ q
x = z ^ q  =  x ← z ⊕ q
```

Logischen Operatoren bedeuten etwas anderes:

$$\texttt{x = !\ z} = x \leftarrow \begin{cases} 1 \text{ if } z = 0 \\ 0 \text{ otherwise} \end{cases}$$

$$\texttt{x = z \&\& q} = x \leftarrow \begin{cases} 1 \text{ if } z \neq 0 \wedge p \neq 0 \\ 0 \text{ otherwise} \end{cases}$$

$$\texttt{x = z || q} = x \leftarrow \begin{cases} 1 \text{ if } z \neq 0 \vee p \neq 0 \\ 0 \text{ otherwise} \end{cases}$$

Typ dieser Ausdrücke ist immer `int`, der Wert immer `0` oder `1`.

ARITHMETISCHE OPERATIONEN

ASSEMBLER

Dank des Zweierkompl. sind diese Ops für (un-)signed gleich

```
add z, q  =  z ← z + q
sub z, q  =  z ← z − q
adc z, q  =  z ← z + q + c
sbb z, q  =  z ← z − q − c
neg z     =  z ← 0 − z        Zweierkomplement
inc z     =  z ← z + 1
dec z     =  z ← z − 1
```

c ist das Carry-Flag, z.B. bei vorherigen Addition gesetzt

C

```
z = q +  p  =  z ← q + p
z = q -  p  =  z ← q − p
z =  -   q  =  z ← 0 − q
z =  ++  q  =  z ← q + 1,   q ← q + 1
z = q ++    =  z ← q,       q ← q + 1
z =  --  q  =  z ← q − 1,   q ← q − 1
z = q --    =  z ← q,       q ← q − 1
```

LINKS- UND RECHTS-SHIFT

Links-Shift um *m* Bits: $\cdot 2^m$, Rechts-Shift um *m* Bits: $\div 2^m$
Im Zweierkomplement mit 4 Bit: $1110b \div 2 = 0111b = 7 \neq -1$
>> mit Sign-Extension heisst «Arithmetischer Rechts-Shift».

SIGN-EXTENSION

TODO:
Wird eine *n*-Bit-Zahl auf eine $n+m$-Bit-Speicherstelle ko-

piert, werden die m oberen Bits auf 0 gesetzt. Bei einer Zahl im Zweierkomplement ändert sich dadurch aber die Bedeutung. Z.B. 4 Bit auf 8 Bit kopieren:

$b = -1 = 1111b \Rightarrow 0000\ 1111b = 15d \neq -1 = 1111\ 1111b$

Statt einer 0 muss überall eine 1 kopiert werden. Dieser Vorgang heisst ***sign extension***: das Vorzeichen wird auf alle zusätzlichen Bits ausgedehnt. Die Sign-Extension wird **nicht** in allen Befehlen automatisch ausgeführt (z.B. `mov`).

ASSEMBLER

Rotates füllen statt mit 0 oder 1 mit ursprünglichen Bits auf.

`shl z, i`	=	$z \leftarrow z \cdot 2^i$	
`shr z, i`	=	$z \leftarrow z \cdot 2^{-i}$	z unsigned
`sar z, i`	=	$z \leftarrow z \cdot 2^{-i}$	z signed
`rol z, i`	=	$z \leftarrow$ Left-Rotate i Bits	
`ror z, i`	=	$z \leftarrow$ Right-Rotate i Bits	

C

`q << p`	=	$z \leftarrow q \cdot 2^p$
`q >> p`	=	$z \leftarrow q \div 2^p$

Beim `>>` wählt Compiler die Instruktion abhängig vom Typ: unsigned: `shr`, signed: `sar`
Rotates können in C nicht direkt ausgedrückt werden.

MULTIPLIKATION

UNSIGNED

Hat nur einen Operanden, der andere ist implizit: `mul z`

Je nach Bit-Grösse von z sind Ergebnis und andere Operand:

Bits	Ergebnis MSBs	Ergebnis LSBs und zweiter Operand
64	RDX	RAX
32	EDX	EAX
16	DX	AX
8	AH	AL

SIGNED

Signed `mul` ⇒ `imul`. Zusätzlich zu Registern bei `mul` gibt es drei für jede Grösse, wo zweiter Operand explizit mitgegeben wird: `imul r, z`. Diese verwenden aber nur LSBs des Ergebnisses

DIVISION

Signed `div` ⇒ `idiv`. Dividend d verwendet dieselben Register wie `mul`, z.B. `RDX:RAX`. Quotient $q = {}^{d}/_{z}$ wird in LSBs von d geschrieben, z.B. `RAX`. Rest $r = d \bmod z$ wird in MSBs von d geschreiben, z.B. `RDX`.

Division in Assembler sollte gemieden werden, da sehr langsame Operation. Sie kann für Zweierpotenzen durch Rechts-Shift und bei anderen Konstanten oft durch Multiplikation oder Umstellen der Formel ersetzt werden.

C

Multiplikation: `a * b`, Division: `a / b`, Rest: `a % b` Die richtige Operation (signed oder unsigned) wird aus Typ abgeleitet Aber: Typ bleibt gleich, d.h. `int * int` → `int`

TODO:
W6 S9..12 (summary S20)

KONTROLLFLUSS

FLAGS

Flags sind einzelne Bits, die in einem gemeinsamen Register `RFLAGS` liegen, das nicht direkt verwendet werden kann.

Carry Flag (CF): Überlauf **unsigned** Arithmetik (Übertrag MSBs)
Overflow Flag (OF): Überlauf **signed** Arithmetik (Vorzeichen stimmen nicht mehr)
Zero Flag (ZF): Wird immer gesetzt, wenn das Resultat `0` ist
Sign Flag (SF): Höchstwertiges Bit des Resultats
Parity Flag (PF): Wenn das niederwertigste Byte des Resultats eine gerade Anzahl an gesetzten Bits enthält

Prozessor weiss nicht, ob signed/unsigned gearbeitet wird
– CF und OF werden immer beide bestimmt
– Das Programm muss später das richtige Flag verwenden

CONDITION CODES (CC)

Zu vielen CCs gibt es den negierten CC mit vorangestelltem `N`

`NC` ⟺ `CF = 0`
`NA` ⟺ `CF = 1 OR ZF = 1`

Beispiel 2: **Conditional Move**

```
dec rax          ; sets ZF = 1 if rax == 0
cmovz rax, [q]   ; sets rax = [q] if rax == 0
```

UNSIGNED

Differenz $a - b$ zeigt an, ob a kleiner, grösser oder gleich b ist:

Fall	Flags	Begründung
$a = b$	ZF = 1	$a - b = 0$
$a < b$	CF = 1	$a - b$ ergibt (unsigned) Überlauf
$a \geq b$	CF = 0	$a \geq b \Leftrightarrow \overline{a < b}$
$a \leq b$	ZF = 1 ∨ CF = 1	$a \leq b \Leftrightarrow a < b \vee a = b$
$a > b$	ZF = 0 ∧ CF = 0	$a > b \Leftrightarrow \overline{a \leq b}$

SIGNED

Fall	Flags	Begründung
$a = b$	ZF = 1	$a - b = 0$
$a < b$	OF ≠ SF	$a < 0 \wedge b > 0 \Leftrightarrow$ OF = 1 ∧ SF = 0 Overflow $a < b \Leftrightarrow$ OF = 0 ∧ SF = 1 No overf.
$a \geq b$	OF = SF	$a \geq b \Leftrightarrow \overline{a < b}$
$a \leq b$	ZF = 1 ∨ OF ≠ SF	$a \leq b \Leftrightarrow a < b \vee a = b$
$a > b$	ZF = 0 ∧ OF = SF	$a > b \Leftrightarrow \overline{a \leq b}$

INTEL CCS

	$a < b$			$a > b$		
signed	Less	L	SF ≠ OF	Greater	G	ZF = 0 ∧ OF = SF
unsigned	Below	B	CF = 1	Above	A	ZF = 0 ∧ OF = 0

CC	Name	Flags	CC	Name	Flags
A	Above	CF = 0 ∧ ZF = 0	AE	A. or Eq	CF = 0
B	Below	CF = 1	BE	B. or Eq	CF = 1 ∨ ZF = 1
G	Greater	ZF = 0 ∧ SF = OF	GE	G. or Eq	SF = OF
L	Less	SF ≠ OF	LE	L. or Eq	ZF = 1 ∨ SF ≠ OF
PE	P. Even	PF = 1	P0	P. Odd	PF = 0
E	Equal	ZF = 1			

JMP

`jmp d` setzt RIP ← RIP + d bezogen auf RIP nach dem Befehl selbst. d ist 8- oder 32-Bit, wird auf 64-Bit sign-extended. Programm springt um d: $d > 0$ ⇒ vorwärts, $d < 0$ ⇒ rückwärts.

Bei Labels berechnet der Assembler die relative Adresse.

VERGLEICHE

`cmp` entspricht `sub` ohne Veränderung der Operanden.
`test` entspricht `and` ohne Veränderung des Ziel-Operanden, setzt aber trotzdem die Flags.

BEDINGTE ANWEISUNGEN

`cmovcc`: Conditional MOV
`jcc`: Conditional JMP
`setcc`: Schreibt 1 in 8-Bit grosses Ziel wenn CC erfüllt, sonst 0

C

Der Body von `if` wird genau dann ausgeführt, wenn die Bedingung nicht `0` ist. Falls ein `else` vorhanden ist, kommt am Ende des ersten Bodys ein unbedingter Sprung.

Beispiel 3:

```
if (ux < 3) {
  // if-body ...
} else {
  // else-body ...
}
```

```
mov eax, [ux]
cmp eax, 2
ja else_body
; if-body
jmp after_if

else_body:
  ; else-body
after_if:
```

SCHLEIFEN

In C ist do-while das Analogon zur Schleife in Assembler:

```
do {
  /* body */
} while (condition);
```

```
loop:
  ; body
  dec rcx
  jnz loop
```

Bei der while-Schleife wird vor jedem Ausführen die Bedingung geprüft, somit ein mal öfter als der Body.

Beispiel 4:

```
int i = 23;
while (--i) {
  /* body */
}
```

```
  mov rcx, 23
  jmp condition
loop:
  ; body
condition:
  dec rcx
  jnz loop
```

POINTER-ADDITION

Die Addition eines Integers zu einem `T*`-Pointer berücksichtigt die Grösse `sizeof(T)` (genauso bei `++`). Z.B. Bei `int *p = (int *)0x20;` ist `p+1 == 0x24` und `p+2 == 0x28`, usw.